분기한정 기법

BRANCH AND BOUND

네 가지 공개 코딩 문제로 배우는 알고리즘

공개 학습 문제는 네 개입니다

6~11장, 챕터별 4문제씩 총 24문제 중 기말에 10문제 출제

번호	문제	난도
8-1	정해진 순회 경로의 비용	아주 쉬움
8-2	남은 작업 비용의 하한	쉬움
8-3	작업을 하나씩 배정하는 최소 비용	기본
8-4	모든 도시를 한 번씩 방문하는 최소 순회 비용	조금 도전

기말은 별도 객관식·서술형 없이 코드 실행 결과로 평가합니다.

최적값을 찾는 도중에도 가지를 줄입니다

현재까지 찾은 가장 좋은 완성 답을 best라고 부릅니다.

분기

다음 후보를 선택해 여러 경우를 탐색합니다.

한정

이 가지에서 얻을 수 있는 비용의 하한을 계산합니다.

가지치기

최선보다 좋아질 수 없는 가지를 더 보지 않습니다.

낙관적인 비용도 안전한 하한이 됩니다

제약 일부를 잠시 풀면 실제 최솟값보다 작아질 수 있습니다.

배정의 제약

각 작업은 한 사람에게만 맡겨야 합니다.

하한 계산

행마다 가장 싼 작업을 고르되 열 중복을 잠시 허용합니다.

계산 결과

이 값은 실제 완성 배정의 최소 비용 이하입니다.

약 2시간 수업 흐름

한 문제를 이해하고 직접 구현하는 시간을 함께 확보합니다.

시간	내용
0–20분	정해진 경로 비용 계산
20–45분	남은 배정 비용의 하한
45–55분	쉬는 시간
55–80분	최소 작업 배정
80–105분	최소 순회 비용
105–120분	직접 구현·경계 테스트

마지막에는 빈 코드에서 다시 작성하고 테스트합니다.

8-1 정해진 순회 경로의 비용

아주 쉬움

1..9개 도시 사이의 이동 비용이 정수 행렬 dist로 주어집니다. 대각선은 0, 나머지 비용은 0..100이며 모든 도시 사이에 이동할 수 있습니다. route는 모든 도시 번호 0..n-1을 한 번씩 포함하고 0에서 시작합니다. route 순서로 이동한 뒤 마지막 도시에서 0으로 돌아오는 총 비용을 반환하세요. 도시가 하나이면 비용은 0입니다.

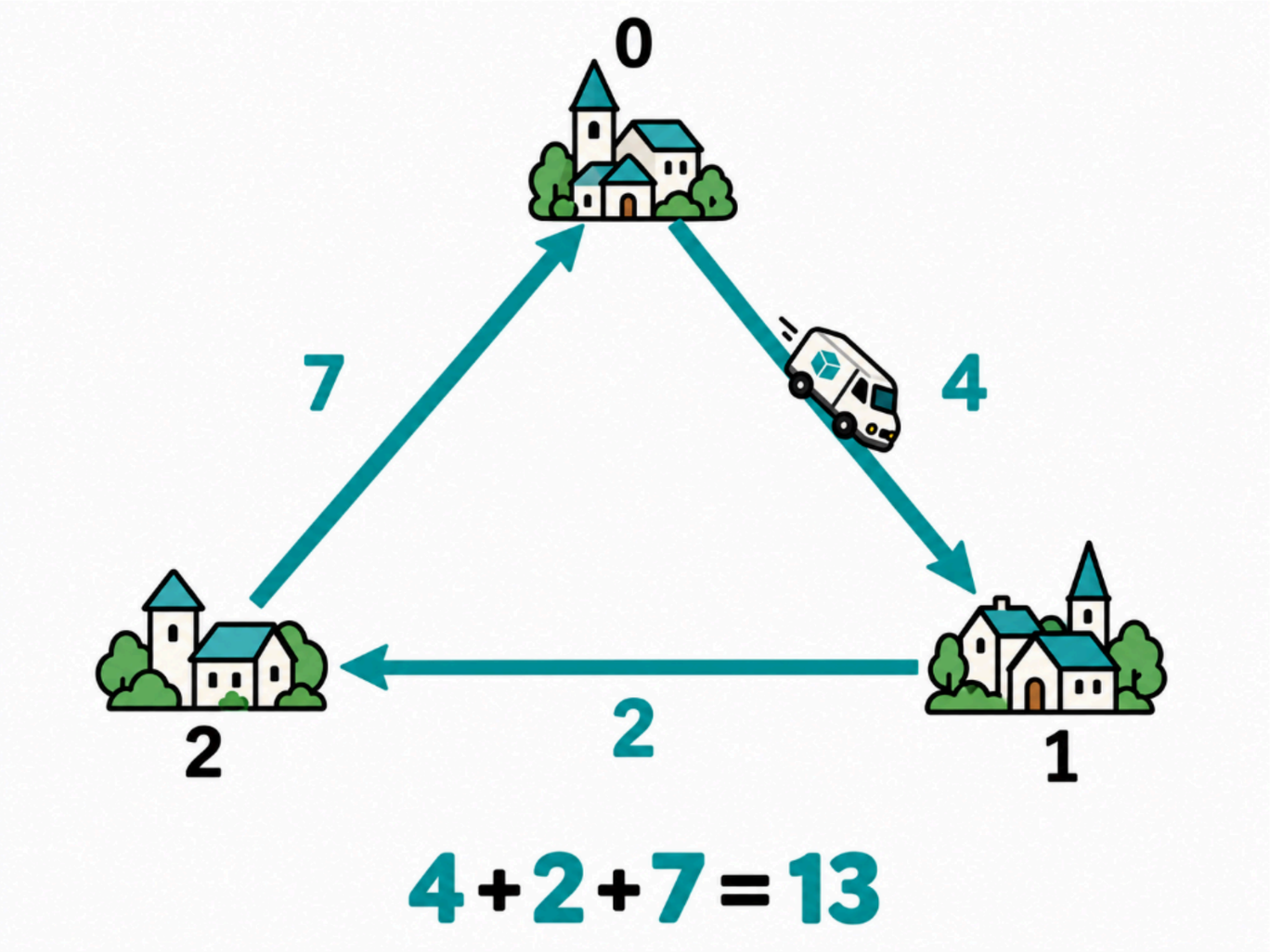
함수와 공개 예제

Python: route_cost(dist, route)

입력	반환값
[[0,4,7],[4,0,2],[7,2,0]], [0,1,2]	13
[[0]], [0]	0

int route_cost(const int dist[][9], int n, const int route[])

정해진 순회 경로의 비용의 작은 예



0, 1, 2, 0
마지막 귀환도 포함
4+2+7=13

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

이동	비용
0에서 1	4
1에서 2	2
2에서 0	7
합계	13

경로의 이웃한 두 도시 사이 비용을 더하고, 마지막 도시에서 출발 도시로 돌아가는 비용도 더합니다.

이 문제의 계산 방법

8-1

경로의 이웃한 두 도시 사이 비용을 더하고, 마지막 도시에서 출발 도시로 돌아가는 비용도 더합니다.

최소 경로를 찾는 문제가 아니라 주어진 경로 하나를 계산합니다. 방향에 따라 비용이 다를 수 있으며 마지막 귀환 비용을 빼먹지 않습니다.

정해진 순회 경로의 비용 · Python

```
def route_cost(dist, route):
    total = 0
    for i in range(len(route)):
        total += dist[route[i]][route[(i+1)%len(route)]]
    return total
```

시간 O(n), 추가 공간 O(1)

정해진 순회 경로의 비용 · C

```
int route_cost(const int dist[][9], int n, const int route[]) {
    int total=0;
    for (int i=0; i<n; ++i)
        total+=dist[route[i]][route[(i+1)%n]];
    return total;
}
```

시간 O(n), 추가 공간 O(1)

경계 입력과 복잡도

8-1

경계 입력

최소 경로를 찾는 문제가 아니라 주어진 경로 하나를 계산합니다. 방향에 따라 비용이 다를 수 있으며 마지막 귀환 비용을 빼먹지 않습니다.

계산량

시간 O(n), 추가 공간 O(1)

직접 작성하고 수정합니다

8-1 정해진 순회 경로의 비용

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

8-2 남은 작업 비용의 하한

쉬움

사람 n명과 작업 n개의 비용 행렬 cost(n=1..8, 비용 0..100)가 있습니다. used[c]는 이미 맡긴 작업이면 1, 아니면 0입니다. 0번부터 next_row−1번 사람까지 배정했고 used의 1 개수는 next_row와 같습니다. 남은 각 사람의 행에서 아직 쓰지 않은 작업의 최소 비용을 구해 더하세요. 이 계산에서는 서로 다른 사람이 같은 작업을 고르는 중복을 잠시 허용합니다. next_row=n이면 0을 반환합니다.

함수와 공개 예제

Python: assignment_bound(cost, used, next_row)

입력	반환값
[[9,2,7],[6,4,3],[5,8,1]], [0,1,0], 1	4
[[4,1],[2,3]], [0,0], 0	3
[[4,1],[2,3]], [1,1], 2	0

int assignment_bound(const int cost[][8], int n, const int used[], int next_row)

남은 작업 비용의 하한의 작은 예

	X	Y
R1	6	3
R2	5	1



3 + 1 = 4

겹침을 잠시 무시한 계산.

남은 두 행에서
각각 가장 싼 값
3+1=4
작업 중복을
잠시 허용합니다.

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

남은 사람	미사용 작업 X	미사용 작업 Y	행의 최솟값
R1	6	3	3
R2	5	1	1
합계			4

아직 쓰지 않은 작업 중 각 행에서 가장 싼 값을 고릅니다. 같은 열을 여러 행에서 골라도 허용하므로 실제 배정의 최솟값 이하인 낙관적 계산입니다.

이 문제의 계산 방법

8-2

아직 쓰지 않은 작업 중 각 행에서 가장 싼 값을 고릅니다. 같은 열을 여러 행에서 골라도 허용하므로 실제 배정의 최솟값 이하인 낙관적 계산입니다.

이 하한 자체는 중복 없는 완성 배정이 아닙니다. 이미 사용한 열은 후보에서 제외합니다. 배정이 모두 끝난 상태의 남은 비용은 0입니다.

남은 작업 비용의 하한 · Python

```
def assignment_bound(cost, used, next_row):
    answer = 0
    for r in range(next_row, len(cost)):
        answer += min(cost[r][c] for c in range(len(cost))
                      if not used[c])
    return answer
```

시간 $O(n^2)$, 추가 공간 $O(1)$

남은 작업 비용의 하한 · C

```
int assignment_bound(const int cost[][8], int n,
                     const int used[], int next_row) {
    int answer=0;
    for (int r=next_row; r<n; ++r) {
        int low=1000000;
        for (int c=0; c<n; ++c)
            if (!used[c] && cost[r][c]<low) low=cost[r][c];
        answer+=low;
    }
    return answer;
}
```

시간 $O(n^2)$, 추가 공간 $O(1)$

경계 입력과 복잡도

8-2

경계 입력

이 하한 자체는 중복 없는 완성 배정이 아닙니다. 이미 사용한 열은 후보에서 제외합니다. 배정이 모두 끝난 상태의 남은 비용은 0입니다.

계산량

시간 $O(n^2)$, 추가 공간 $O(1)$

직접 작성하고 수정합니다

8-2 남은 작업 비용의 하한

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

10분 쉬었다가 이어갑니다

앞의 두 계산을 정리합니다.

뒤의 문제는 앞에서 익힌 계산을
반복하거나 탐색에 연결합니다.

8-3 작업을 하나씩 배정하는 최소 비용

기본

사람 n명과 작업 n개의 비용이 n×n 행렬 cost로 주어집니다. n=1..8, 각 비용 0..100입니다. 각 사람에게 작업 하나를 배정하고 각 작업도 한 사람에게만 맡겨야 합니다. 사람 r이 작업 c를 맡을 때 cost[r][c]를 더합니다. 전체 비용의 최솟값을 반환하세요.

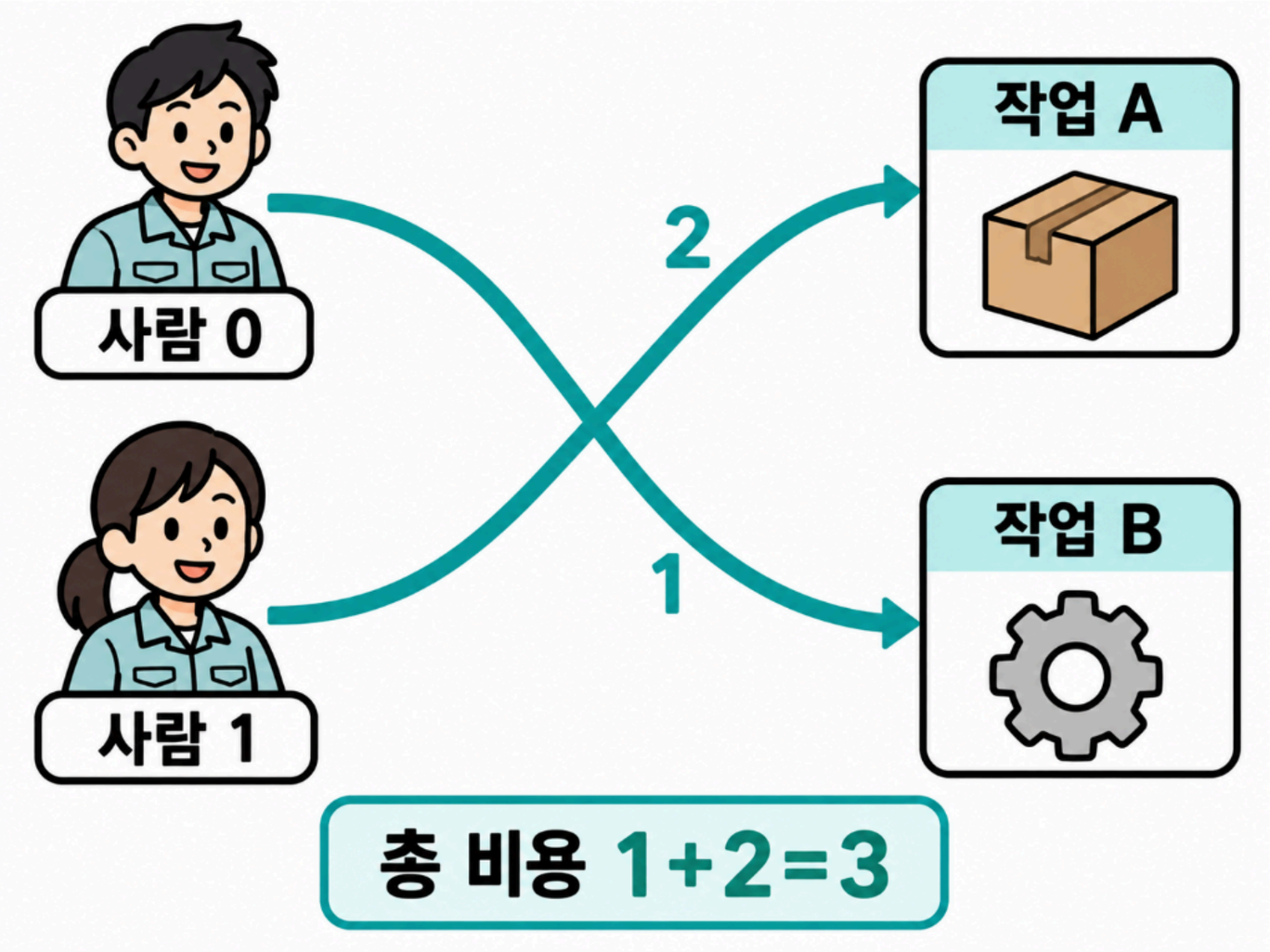
함수와 공개 예제

Python: assignment_min(cost)

입력	반환값
[[4,1],[2,3]]	3
[[7]]	7
[[9,2,7],[6,4,3],[5,8,1]]	9

int assignment_min(const int cost[][8], int n)

작업을 하나씩 배정하는 최소 비용의 작은 예



사람0은 작업B
사람1은 작업A
총 비용은
1+2=3입니다.

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

사람0의 작업	사람1의 작업	총 비용
A: 4	B: 3	7
B: 1	A: 2	3

사람을 한 명씩 정하고 아직 사용하지 않은 작업을 시도합니다. 현재까지의 비용이 이미 찾은 완성 배정보다 작아질 수 없으면 그 가지를 중단합니다.

이 문제의 계산 방법

8-3

사람을 한 명씩 정하고 아직 사용하지 않은 작업을 시도합니다. 현재까지의 비용이 이미 찾은 완성 배정보다 작아질 수 없으면 그 가지를 중단합니다.

모든 비용이 0 이상이므로 앞으로 선택을 더해도 현재 비용은 줄지 않습니다. used를 선택 뒤에 다시 0으로 복구해야 다른 배정도 탐색합니다.

작업을 하나씩 배정하는 최소 비용 · Python

```
def assignment_min(cost):
    n = len(cost)
    used = [False]*n
    best = 10**9
    def search(row, total):
        nonlocal best
        if total >= best: return
        if row == n:
            best = total
            return
        for col in range(n):
            if not used[col]:
                used[col] = True
                search(row+1, total+cost[row][col])
                used[col] = False
    search(0, 0)
    return best
```

보수적 최악 시간 O(n·n!), 공간 O(n)

작업을 하나씩 배정하는 최소 비용 · C

```
static int assign_go(const int cost[][8], int n, int row,
                     int used[], int total, int best) {
    if (total>=best) return best;
    if (row==n) return total;
    for (int c=0; c<n; ++c) if (!used[c]) {
        used[c]=1;
        best=assign_go(cost,n,row+1,used,total+cost[row][c],best);
        used[c]=0;
    }
    return best;
}
int assignment_min(const int cost[][8], int n) {
    int used[8]={0}; return assign_go(cost,n,0,used,0,1000000000);
}
```

보수적 최악 시간 O(n·n!), 공간 O(n)

경계 입력과 복잡도

8-3

경계 입력

모든 비용이 0 이상이므로 앞으로 선택을 더해도 현재 비용은 줄지 않습니다. used를 선택 뒤에 다시 0으로 복구해야 다른 배정도 탐색합니다.

계산량

보수적 최악 시간 O(n·n!), 공간 O(n)

직접 작성하고 수정합니다

8-3 작업을 하나씩 배정하는 최소 비용

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

8-4 모든 도시를 한 번씩 방문하는 최소 순회 비용

조금 도전

도시 n개(n=1..9)의 이동 비용 행렬 dist가 있습니다. 대각선은 0, 각 비용은 0..100이며 모든 도시 사이에 이동할 수 있습니다. 0번에서 출발해 다른 도시를 각각 한 번 방문하고 0번으로 돌아옵니다. 가능한 순회 중 총 비용의 최솟값을 반환하세요. 방향에 따라 비용이 다를 수 있고, n=1의 답은 0입니다.

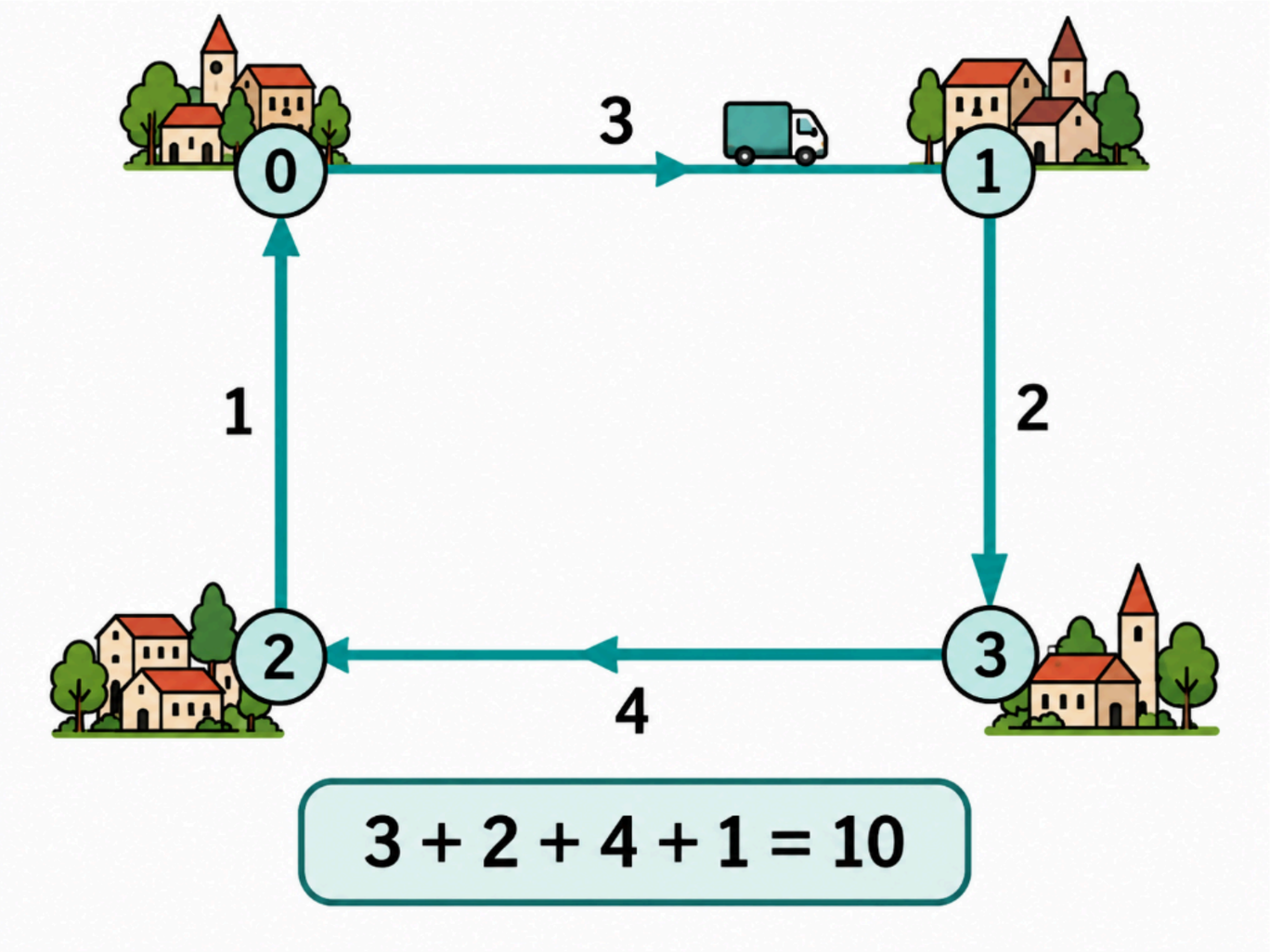
함수와 공개 예제

Python: tsp_min(dist)

입력	반환값
[[0,4,7],[4,0,2],[7,2,0]]	13
[[0]]	0
[[0,3,1,8],[3,0,8,2],[1,8,0,4],[8,2,4,0]]	10

int tsp_min(const int dist[][9], int n)

모든 도시를 한 번씩 방문하는 최소 순회 비용의 작은 예



0, 1, 3, 2, 0
모든 도시를 방문
출발지로 돌아오면
3+2+4+1=10

작은 입력을 손으로 계산합니다

가능한 선택이나 중간 계산을 차례대로 확인합니다.

순회	비용
0,1,2,3,0	3+8+4+8=23
0,1,3,2,0	3+2+4+1=10
0,2,1,3,0	1+8+2+8=19
최솟값	10

방문하지 않은 다음 도시를 하나씩 선택합니다. 모든 도시를 방문하면 0번으로 돌아오는 비용까지 더해 현재 최선과 비교합니다.

이 문제의 계산 방법

8-4

방문하지 않은 다음 도시를 하나씩 선택합니다. 모든 도시를 방문하면
0번으로 돌아오는 비용까지 더해 현재 최선과 비교합니다.

마지막 귀환 비용을 반드시 더합니다. 0번은 처음에 방문 표시합니다.
비용이 음수가 없어서 현재 합이 최선 이상이면 더 내려갈 필요가 없습니다.

모든 도시를 한 번씩 방문하는 최소 순회 비용 · Python

```
def tsp_min(dist):
    n = len(dist)
    used = [False]*n
    used[0] = True
    best = 10**9
    def search(last, count, total):
        nonlocal best
        if total >= best: return
        if count == n:
            best = min(best, total+dist[last][0])
            return
        for nxt in range(1, n):
            if not used[nxt]:
                used[nxt] = True
                search(nxt, count+1, total+dist[last][nxt])
                used[nxt] = False
    search(0, 1, 0)
    return best
```

보수적 최악 시간 O(n·(n−1)!), 공간 O(n)

모든 도시를 한 번씩 방문하는 최소 순회 비용 · C

```
static int tour_go(const int d[][9], int n, int last,
                   int count, int used[], int total, int best) {
    if (total>=best) return best;
    if (count==n) {int v=total+d[last][0]; return v<best?v:best;}
    for (int x=1; x<n; ++x) if (!used[x]) {
        used[x]=1;
        best=tour_go(d,n,x,count+1,used,total+d[last][x],best);
        used[x]=0;
    } return best;
}
int tsp_min(const int dist[][9], int n) {
    int used[9]={0}; used[0]=1;
    return tour_go(dist,n,0,1,used,0,1000000000);
}
```

보수적 최악 시간 O(n·(n−1)!), 공간 O(n)

경계 입력과 복잡도

8-4

경계 입력

마지막 귀환 비용을 반드시 더합니다. 0번은 처음에 방문 표시합니다. 비용이 음수가 없어서 현재 합이 최선 이상이면 더 내려갈 필요가 없습니다.

계산량

보수적 최악 시간 $O(n \cdot (n-1)!)$, 공간 $O(n)$

직접 작성하고 수정합니다

8-4 모든 도시를 한 번씩 방문하는 최소 순회 비용

① 예상 반환값을 적습니다.
② 빈 코드에서 함수를 작성합니다.
③ 공개 예제와 경계 입력으로 실행합니다.
④ 틀린 입력을 고치고 다시 제출합니다.

이 장의 네 문제 정리

강의노트와 사이트는 같은 함수 규격을 사용합니다.

문제	함수	난도
8-1	route_cost	아주 쉬움
8-2	assignment_bound	쉬움
8-3	assignment_min	기본
8-4	tsp_min	조금 도전

공개 문제의 조건 안에서 올바른 반환값을 구합니다.

기말 준비

공개 24문제 중 10문제 출제

6~11장, 챕터마다 네 문제를 공부합니다.
정해진 함수 인자와 반환값을 지킵니다.
직접 작성한 코드의 실행 결과로 평가합니다.

분기한정 기법 정리

BRANCH AND BOUND

8-1 정해진 순회 경로의 비용
8-2 남은 작업 비용의 하한
8-3 작업을 하나씩 배정하는 최소 비용
8-4 모든 도시를 한 번씩 방문하는 최소 순회 비용